

王帅俭

18614236639 | a978685835@163.com | 男 | 12 年经验 | 1990.10

核心优势

- 技术栈全面：10 年 Golang / 3 年 C++ / 3 年 Linux 开发经验，深耕云计算（Kubernetes、GPU 虚拟化）、大模型训练优化（Megatron、MoE）、高性能调度（Binpack、拓扑亲和）。
- 工程能力强：主导多个公司级 AI 平台架构设计（百度 AI 平台、Shopee MLP 调度），解决 GPU 利用率、容错、混布等核心问题。
- 性能经验：实现 DP Overlap、Memory-Cost Model 等优化策略，提升大模型训练 MFU（Model FLOPs Utilization）。

工作经历

2025.06 - 至今

马上消费金融股份有限公司

资深架构师

计算机软件 | 民营 | 规模：1000-9999 人

工作描述：大模型预训练 — 训练框架、算法创新与性能优化

- 性能基准测试：完成 B200 等多型号 GPU 在 Dense 与 MoE 模型下的训练性能基准测试，产出 MFU/吞吐/显存占用对比数据，为硬件选型与集群容量规划提供决策依据
- 稳定性：基于基建环境差的问题，改造 nccl，将 topo 和 channel 可视化，快速定位环境问题
- 预训练算法创新 Multi-LM-Head：完成 Multi-LM-Head 预训练方案，打通模型改造与训练、loss 曲线分析、评测等流程，验证新结构在预训练阶段的效果与收益。
- Megatron 训练疑难问题排查：定位并修复 Megatron dataset 使用中的爆内存问题；深入分析 validation loss 反常低于 training loss 的根因，保障训练指标的可解释性与可信度；在 Megatron 中实现训练 step 与数据样本的精确映射，loss spike 发生时可快速回溯定位异常样本，大幅提升数据质量排查与训练稳定性分析的效率。

2023.12 - 2025.05

虾皮（Shopee）

研发工程师

计算机软件 | 民营 | 规模：1000-9999 人

工作描述：AI 基础架构 — 调度与训练优化

- GPU 调度系统：设计单 GPU Pod Group Binpack 算法，降低资源碎片率 30%；实现 NCCL 拓扑亲和调度；实现共享 GPU 调度并指定 GPU 卡，提升 GPU 利用率 10%。
- 稳定性提升：开发 Torch 作业容错框架，结合 NCCL 日志分析工具，故障排查效率提升 50%。
- 新技术调研：调研 MoE（混合专家模型）在分布式训练中的落地可行性分析。

2021.04 - 2023.12

百度

Golang 研发工程师

计算机软件 | 民营 | 规模：1000-9999 人

工作描述：AI 平台建设（孔明），GPU 虚拟化，大模型训练

- 大模型性能优化：迁移 ChatGLM-6B 至 Megatron 框架，重构 RoPE；通过 DP Overlap 技术提升 MFU。
- GPU 虚拟化：主导 CGPU 算力/显存隔离方案，实现故障隔离与透明迁移，利用率提升 35%。
- 调度系统：设计共享 GPU 调度方案（Volcano 集成），支持超发抢占与混布，集群资源利用率达 65%。

2020.10 - 2021.03

猿辅导

Golang 研发工程师

计算机软件 | 民营 | 规模: 1000-9999 人

工作描述: 负责基础架构 Kubernetes 平台建设。

2018.01 - 2020.10

北京京东 (T6)

Golang 研发工程师

计算机软件 | 民营 | 规模: 1000-9999 人

工作描述: 前期参与 Baas 平台建设, 围绕 Kubernetes 结合区块链业务构建区块链底层平台, 负责指导、处理、协调和解决团队云计算方面的技术问题; 后期负责商城 Paas 平台虚拟网络建设。

2016.07 - 2017.12

北京东方国信科技股份有限公司

Golang 研发工程师

计算机软件 | 民营 | 规模: 1000-9999 人

工作描述: 参与 Paas 平台建设, 围绕 Kubernetes 结合自身业务做增强功能, 负责指导、处理、协调和解决公司云计算项目中出现的技术问题。

2013.07 - 2016.06

山东金码信息技术有限公司北京分公司

软件研发工程师

计算机软件 | 民营 | 规模: 1000-9999 人

工作描述: 根据需求编写代码, 并参与软件开发的全过程, 包括需求分析、环境部署、软件开发、系统上线和一些维护工作。

项目经历

2025.06 - 至今 大模型预训练 (马上消费金融)

项目背景: 公司引入 B200 新硬件并启动大模型预训练建设, 需完成新硬件性能评估、探索模型结构创新方向, 同时保障训练框架在规模化场景下的稳定性与可观测性。

责任描述: 负责新硬件性能评估与大模型预训练创新方案端到端落地, 覆盖数据、训练、评测、问题排查全链路。

性能基准测试

- 基于 Megatron 在 B200 上搭建完整评测环境, 基于 Qwen3-8B 和 qwen3-30B 跑通 Dense 与 MoE 两类模型, 发现 B200 dense 性能符合预期, 但是 moe MFU 上不去, 经过调整性能跟阿里云对齐
- 产出 MFU、吞吐 (tokens/s) 等核心指标, 与 H100 对比形成横向数据报告, 为公司后续硬件选型、集群容量规划与成本测算提供决策支撑。
- 对于基建差的问题, 改造 nccl, 在初始化阶段将检测到的设备和 topo 打印出来, 将 channel 信息记录到文件中, 通过 web 图形化展示, 可以一眼看出当前硬件或者配置的问题

预训练算法创新: Multi-LM-Head

- 完成 Multi-LM-Head 预训练方案从 0 到 1 验证: 主要包括模型结构改造、跑 loss 曲线, 评测等过程。
- 在 Megatron 框架内实现多 LM Head 结构, 尝试利用 kl 学习最后一层 lm-head, 利用余弦相似度学习最后一层 hidden-state, 添加线性层等多种方式、编写算子优化 lm-head 显存占用提升训练性能, 性能提升 30%。
- 对比 baseline 细粒度分析 loss 曲线 (train/val loss、各 head loss 分解) 与下游评测指标, 验证新结构在预训练阶段的效果与收益, 为后续策略选型提供实验依据。

Megatron 训练疑难问题排查

- 定位并修复 Megatron dataset 使用中的爆内存问题: 通过内存 profile 定位到一个数据集文件对一个 mask, 数据集文件太多的话, 内存线性增长, 优化后消除训练启动阶段的内存瓶颈。
- 深入分析 validation loss 反常低于 training loss 的根因, 排查到数据采样与验证集构造等环节的差异, 给出修复方案, 保障训练指标的可解释性与可信度。

- 在 Megatron 中实现训练 step 与数据样本的精确映射，记录索引信息，保证分布式训练下的可追溯性，loss spike 发生时，支持从异常 step 快速回溯到触发该 spike 的具体样本。

2023.12 - 2025.05 AI 基础架构 — 调度与训练优化（虾皮）

项目背景：公司 AI 训练规模快速扩张，GPU 资源紧张，作业稳定性与集群利用率面临持续压力，需在调度层与训练框架层系统性优化。

责任描述：负责 GPU 调度与训练框架相关工作，提升集群利用率、作业稳定性，并推进新技术方向调研落地。

核心调度功能

- 实现单 GPU Pod Group Binpack 调度算法，从卡、拓扑多维度优化资源分配策略，有效降低资源碎片率 30%，主要是将作业分成多个组，一个组放到一个节点上，可以理解为为了适配 TP，要把完整的一个组放到一节点上，这样可以提高训练性能；同时兼容单机多卡、多机多卡等多种训练场景。
- 实现拓扑亲和调度：分配卡时优先选择距离较近的组合（同轨 GPU，交换机，NVLink/PCIe 亲和），减少跨 NUMA、跨 PCIe switch 的通信开销，提升通信密集型作业的训练效率。
- 实现共享 GPU 调度并支持指定 GPU 卡，为不同资源粒度的作业提供统一调度入口，GPU 利用率提升 10%。

稳定性建设

- 开发 Torch 作业容错框架：修改 torch 框架，因为容错 etcd 跟 k8s 不适配，掉卡之类的只是会在内容重启，不会让 Pod 感知到，因此只要出现错误就要结束管理进程，退出 Pod，然后控制器再拉起新的 Pod；捕获节点/卡级别故障并自动恢复训练，减少人工介入；结合 NCCL 日志分析工具，故障排查效率提升 50%。
- 外围增加系统故障检查排查能力（节点健康、网络、NCCL 异常等），与容错机制联动形成闭环。
- 在 NCCL 侧增加关键路径日志，构建集群拓扑信息可视化，直观看到 GPU、网卡最终选择路径，快速判断是否影响性能。

利用率提升与新技术调研

- 解决新机型拓扑带来的性能问题，通过挂载 NCCL 拓扑以及指定卡号等手段进一步压榨硬件收益。
- 调研 MoE（混合专家模型）在分布式训练中的落地可行性，包括通信模式、负载均衡、Expert Parallel 方案等，输出调研结论与可行性分析。
- 调研强化学习训练在分布式环境下的工程挑战，为后续技术选型提供参考。

2023.05 - 2023.12 大模型项目（百度）

项目背景：团队需在 Megatron 框架上跑通 ChatGLM 系列模型并做深度性能调优，沉淀大模型训练性能优化方法论。

责任描述：负责模型迁移与训练性能优化，提升 MFU。

- 迁移 ChatGLM-6B 到 Megatron 框架：适配模型结构差异（RoPE 位置编码、LayerNorm 实现、attention mask 等），逐层对齐 tensor 精度，确保迁移后训练行为与原实现一致。
- 调研 activation checkpointing 重计算方案，优化 Megatron 中重计算现有逻辑，减少无效重计算开销。
- 构造 Memory-Cost Model：通过模拟数据推算不同并行切分方式（TP/PP/DP 组合）下的显存占用与通信开销，辅助选择最优并行策略，避免线上试错成本。
- 在 Megatron 上实现 DP Overlap（梯度 allreduce 与反向计算重叠），使用 nsys 分析通信/计算时间线验证优化效果，MFU 显著提升。
- 日常跑各种模型规模及相关特性（FlashAttention、重计算、并行策略等）对应的 MFU

2022.10 - 2023.04 GPU 虚拟化（百度）

项目背景：公司 AI 训练作业对 GPU 资源需求多样化（推理/训练/开发），整卡粒度分配浪费严重，需要细粒度

切分与资源隔离能力。

责任描述：负责 CGPU（算力/显存隔离）研发、GPU 池化研发、作业迁移功能。

CGPU 算力/显存隔离

- 维护 CGPU 核心功能，持续优化算力和显存的隔离精度与用户体验。
- 保障故障隔离能力：单个作业异常不影响同卡其他作业；减少高低优先级作业避让带来的抖动问题。
- GPU 整体利用率提升 35%。

透明容错 / 作业迁移

- 设计并实现透明容错方案：通过训练框架与底层运行时协同，故障节点上的作业可无感迁移到健康节点，用户侧感知最小化。

GPU 池化

- 负责 GPU 池化项目，自底向上打通多租户和算力应用，构建资源池抽象层。
- 实现公有云场景下 GPU 的任意切分使用，按需分配算力与显存，支持弹性伸缩与跨机使用。

2022.01 - 2022.10 AI 平台（孔明）（百度）

项目背景：孔明 AI 平台需进一步提升 GPU 利用率，支持在离线混布、VPA 等调度新特性，应对业务多样化资源需求。

责任描述：负责离线调度团队，主导离线训练 VPA、修复共享 GPU 调度方案，以及增加在离线混布、支持编解码等调度功能。

- 修复共享 GPU 调度方案：由于 kubelet 没有把 Pod 信息传递到 device plugin，现有共享方案可能出现重复分配卡的情况；通过在 runtime 层修改修正分配卡信息，根治重复分配问题，保障共享 GPU 调度正确性。
- 离线训练 VPA：为了进一步提升 GPU 利用率，通过利用率动态将 GPU 使用资源缩减到 1/2，可混布其他作业进一步提升 GPU 利用率（当时 K8s 社区也在支持该功能，实现还不稳定，未全量上线，但为后续 VPA 方向积累了工程经验）。
- 在离线混布与编解码调度：开发多种调度策略，在离线混布采用卡粒度亲和性减少资源争抢；编解码场景增加两种资源类型，支持更细粒度的异构资源调度。

2021.04 - 2021.12 AI 平台（孔明）调度（百度）

项目背景：为了提升 GPU 利用率、提高作业运行质量，需在调度层面补齐容错弹性、超发抢占、共享 GPU 调度等核心能力。

责任描述：设计并实现离线训练容错弹性、超发抢占及共享 GPU 调度方案，并推全上线。

- 支持容错弹性训练：调度侧实现 EDL（Elastic Deep Learning）训练，自定义作业对象（GRD），在控制器实现作业 Pod 的扩缩，维护完整状态机，训练过程中可根据资源状况和故障情况动态调整参与训练的节点数。
- 支持共享 GPU 调度方案：由于升级问题，从魔改原生调度器改为使用 Volcano 调度器，并在 Volcano 中实现共享 GPU 调度方案，打通整卡、NVIDIA 原生资源表达等多种 GPU 资源形式的统一调度。
- 支持作业超发功能：超发作业采用资源借用模式，不从队列中扣减资源；同时超发作业优先级最低，队列资源不足时通过抢占驱逐超发作业，在保障正常作业 SLA 的同时最大化利用空闲资源，集群资源利用率达 65%。

2020.10 - 2021.03 K8s 平台建设（猿辅导）

项目背景：公司基础架构 Kubernetes 平台处于建设早期，需补齐统一认证、故障自愈等周边能力，保障平台稳定性。

责任描述：规划网络方案，对接其他组件，完善平台可用性。

- 对接 idaas 统一认证系统：以 client-go-credential-plugin 方式实现 authn/authz 服务，让 K8s 用户无感接入公司统一认证体系。
- 利用 NPD (Node Problem Detector) 构建故障处理系统，主要针对 docker / BGP / POD 等问题，将事件统一入库并接入告警，形成节点健康闭环。

2020.04 - 2020.09 商城 Paas 平台建设（京东）

项目背景：商城核心业务容器化落地过程中，对 Docker 稳定性、安全隔离、资源观测等提出更高要求。

责任描述：定制 Docker、调研 Kata、构建故障处理系统。

- 在 Docker 上做深度定制化工作：升级 Docker 版本，修复 dockerhang 等生产问题，定制业务需要的功能参数。
- 针对大流量高负载下 Docker 隔离缺陷暴露的问题，对 Kata Containers 进行调研，评估轻量虚拟化方案的性能与安全收益，并在生产环境落地使用。
- 采用 Webhook 等多种方式实现 lxcfs 跟 K8s 结合，完成容器内部资源指标（CPU/内存等）的视角隔离，解决容器内看到宿主机资源信息的问题。

2019.08 - 2020.03 商城 Paas 平台虚拟网络建设（京东）

开发工具：JetBrains、Golang

项目背景：基于每年 618、双 11 大促需求，必须保证线上网络服务稳定；同时私有云需要与京东云/腾讯云等公有云打通，支持混合云部署。

责任描述：接手 BGP 网络项目，开发新的网络项目。

- 接手 BGP 网络项目后梳理整体流程、修补历史遗留漏洞、增加平台化功能，为大促保驾护航。
- 独立完成私有云平台上京东云项目，调用京东云/腾讯云 API 接口，实现平台网络组件与公有云 VPC/路由的适配，支持混合云网络打通。

2019.04 - 2019.08 Baas 平台二期（京东）

开发工具：JetBrains、Golang

项目背景：Fabric 项目只提供二进制工具管理 peer，企业用户使用门槛高，平台侧需要抽象出更好用的 peer 管理能力。

责任描述：负责 Fabric 优化及 peer 管理平台化。

- 为平台增加节点管理功能，使用户可方便地为该组织下的部门或子组织设置节点，降低 Fabric 使用门槛。
- 设计方案：调用 Kubernetes 接口创建 deployment 等 peer 相关资源；利用 Informer 机制监视 deployment 资源状态变化；通过事件回调触发数据库状态改写，完整覆盖 peer 全生命周期管理。

2019.03 - 2019.04 Baas 上京东云二期（京东）

开发工具：JetBrains、Golang

项目背景：一期仅内部用户试用，二期要公测上线，需补齐订单计费模块与 K8s 版本升级等关键能力。

责任描述：完成订单计费、Kubernetes 版本升级测试及监控功能；负责订单计费及 K8s 升级测试。

- 为公测上线补齐订单计费模块：双写事件分发机制保证后台前端监控等下游服务顺利接入；采用状态机维护订单全生命周期状态，高效集成订单计费模块。
- 完成京东云 K8s 团队版本升级测试：覆盖核心 API 兼容性、工作负载迁移验证，升级后平台更稳定。
- 联调过程中识别并规避 POC 阶段未发现的 API 接口风险（通过手动改数据库临时修复），未影响测试和上线进度。

2019.01 - 2019.02 stellar 底层集成（京东）

开发工具：JetBrains、Golang

责任描述：负责 stellar 网络的部署集成与 API 封装。

- 编写 Helm Chart 完成 stellar 网络的一键启动，解决多节点编排与配置管理问题。
- 封装 stellar API 为 rtmc 模块提供统一接口，支持发行资产、转账等业务操作，屏蔽底层链路细节。

2018.10 - 2018.12 Baas 上京东云一期（京东）

开发工具：JetBrains、Golang

项目简介：Baas 支持混合云，利用京东云作为公有云部分融合进公司私有云服务。

责任描述：上云主要包括账户打通、服务迁移适配（Helm Chart 及 Kubernetes 相关功能适配）、京东云资源及 Kubernetes API 调用。

- 主要负责调用 API 对京东云资源及 Kubernetes 集群进行统一管理，屏蔽公有云与私有云的差异。
- 由于安全合规需要，实现验证码功能，增强用户身份校验。

2018.06 - 2018.08 Baas 平台存储模块（京东）

开发工具：JetBrains、Golang

项目背景：Fabric 的公用配置文件与区块链数据有共享存储与自动扩容诉求，避免手工同步和存储打满。

责任描述：带两位同事负责存储模块开发，为区块链服务提供共享存储支持。

- 将 Fabric 公用配置文件统一放到共享存储，避免多节点同步之苦。
- 基于 glusterfs 及 Kubernetes storageclass 扩容机制，上层加一层控制层封装存储模块业务语义。
- 后期发现 glusterfs 效率较慢，集成 rook 安装 Ceph 作为新的后端存储方案，性能显著提升。

2018.01 - 2018.08 Baas 平台一期（京东）

开发工具：JetBrains、Golang、Shell

项目简介：Baas 平台主要结合 Kubernetes 及区块链作为底层平台，为上层业务提供服务。

项目背景：业界当时并无成熟的区块链底层平台，区块链开发部署繁琐、学习成本高，Fabric 跟容器天然亲和，Baas 平台可以显著降低研发人员及公司接入区块链服务的门槛。

责任描述：参与 Baas 架构设计，向团队普及 Kubernetes 基本套路及用法；平台搭建起来后，参与一键部署及企业组网开发工作。

- 后端架构：采用 rancher + Kubernetes + Helm + 应用商店 的模式搭建后端服务，快速交付 MVP。
- 前端：仿照 rancher 页面风格做 Baas 前端，降低用户学习成本。
- DevOps：利用 Jenkins + Helm 仓库 + Docker 仓库 的模式搭建开发/发布环境。
- Fabric Helm Chart 编写完成后，拆分组网操作步骤，抽象出一键部署与企业组网流程，让业务方分钟级上链。

教育经历

2009.09 - 2013.06

计算机科学与技术 / 本科

山东科技大学